Temporal Machine Learning for In-Game Detection of Major League Baseball Pitcher Fatigue from Statcast Pitch Data

Braeden Mefford
M.S. Data Science and Analytics
College of Charleston
DATA 534: Applied Machine Learning

***Abstract*—Pitcher fatigue in Major League Baseball drives both performance decline and injury risk, yet the decision to remove a starting pitcher still relies on round number pitch counts and the visual judgment of the pitching coach. This report presents a pitch by pitch supervised classifier that flags fatigue from MLB Statcast tracking data alone. Seven regular seasons of data from 2019 through 2025 were obtained through the public `pybaseball` interface, yielding 4,603,293 pitches across 93,942 pitcher game appearances. Over thirty rolling window features were engineered to capture velocity decay, spin decay, release point drift, command, and rising contact quality. A leak free forward looking label was constructed by requiring both a fastball velocity drop of at least 1.5 mph and a hard contact rate rise of at least ten percentage points in the next twenty pitches relative to a per game baseline. Three models were compared on a temporal split that held the 2024 and 2025 seasons out for testing: logistic regression, random forest, and gradient boosted trees through XGBoost. XGBoost was the best model with an ROC-AUC of 0.881, a PR-AUC of 0.271, and an F1 of 0.335 at the F1 optimal threshold. Feature importance analysis identified rolling hard contact rate, fastball velocity decay, pitch count, fastball velocity slope, and rolling spin slope as the leading indicators of in game fatigue. A simulated case study shows that the model raises probability above the decision threshold roughly fifteen pitches before opponent performance visibly collapses, which is enough lead time for a coach to prepare a relief plan.**

***Index Terms*—Major League Baseball, pitcher fatigue, sabermetrics, Statcast, time series features, gradient boosting, classification, sports analytics**

I. Introduction and Background

Pitcher fatigue is a central operational problem in Major League Baseball. Once a starting pitcher tires, fastball velocity drops, command degrades, and opposing hitters generate harder contact. The same fatigue is also a leading mechanism for elbow and shoulder injuries, including ulnar collateral ligament reconstruction, commonly known as Tommy John surgery. Mastroianni et al. recently showed that MLB pitchers who go on to require this reconstruction display measurable changes in pitch tracking metrics in the months leading up to the procedure, with side movement, raw speed, and spin all carrying injury risk [7]. Pluciński et al. demonstrated that a classifier trained on routinely captured pitch data can flag elevated injury risk up to one hundred days before surgery [8]. Pitcher arms are large financial assets, and a single torn ligament can cost a season or more.

Despite the stakes, the in game decision to remove a pitcher is still driven by heuristics. Phillips analyzed two decades of MLB substitution data and found that managers replace starters at round pitch counts ending in zero significantly more often than at counts ending in eight or nine, with no underlying performance difference to justify the pattern [1]. Heuristic pitcher removal contrasts with the depth of the data the league already collects. MLB Statcast logs roughly seven hundred thousand pitches per season at the pitch level, with velocity, spin, release point, movement, plate location, and batted ball outcome captured for every pitch [12]. The data is rich enough to support a real time, objective fatigue model.

The biomechanical evidence that fatigue manifests in measurable ways is solid. Grantham et al. filmed eleven NCAA Division I starters with high speed cameras and found that hip lean, elbow height at foot contact, and shoulder rotation all change as pitch counts climb during an inning and across the season, with kinematic shifts that align with subjective fatigue measures [2]. Whiteside et al. showed at MLB scale that release point variability, ball speed, and pitch speed variation predict fielding independent pitching performance, and they later established that release speed and release point both decline as the season progresses [3], [4]. The signals that biomechanics studies pick up in the lab show up in the Statcast tracking feed.

Several authors have used these signals for managerial decision support. Woodham et al. fit pitcher specific regularized linear models to predict whether a starter should be pulled and reported that their model would have changed the decision in 48 percent of starts, with the pitcher performing poorly 60 percent of the time when the manager left him in but the model would have removed him [5]. Kabra used K-means clustering to sort pitchers into eight archetypes from a single 2021 season and concluded that spin rate decline, not velocity, was the dominant fatigue signal [6]. Beyond fatigue specifically, machine learning is now widely applied across baseball analytics, with recent surveys cataloging supervised, unsupervised, and reinforcement learning approaches in sabermetrics [9].

This project builds on the prior work in three directions. First, it covers seven seasons of Statcast rather than one, which is roughly a six fold increase in data. Second, it engineers thirty plus temporal features across velocity, spin,

release point, movement, command, and contact quality, where prior work used a small handful of inputs. Third, and most importantly, it uses a forward looking, leak free fatigue label based on the next twenty pitches and trains a supervised classifier that produces a probability for every pitch in real time. The deliverable is a per pitch fatigue probability that the model produces in real time and with enough lead time before performance collapses to be acted on by a coach.

II. Dataset Description

The data source is MLB Statcast, the radar and optical tracking system installed in every MLB ballpark. Statcast records the speed, spin, release point, movement, and plate location of every pitch, along with batted ball exit velocity, launch angle, and Statcast derived expected statistics such as expected weighted on base average (xwOBA) and expected batting average (xBA) [12]. The data was retrieved through `pybaseball`, an open source Python package that wraps the Baseball Savant search endpoint [13].

Seven regular seasons were pulled from 2019 through 2025, covering a total of 4,603,293 pitches across 93,942 distinct pitcher game appearances. The 2020 season was shortened by COVID, and a per year date window dictionary was used so that the shortened season was handled correctly. The columns retained per pitch include identifiers (game, pitcher, batter, inning, at bat number, pitch number), pitch classification (pitch type and pitch name), velocity (`release_speed` and `effective_speed`), spin (`release_spin_rate`, `spin_axis`), release point (`release_pos_x`, `release_pos_y`, `release_pos_z`, `release_extension`), movement (`pfx_x`, `pfx_z`), plate location (`plate_x`, `plate_z`, zone), description and outcome, and batted ball tracking (`launch_speed`, `launch_angle`, `bb_type`, `barrel`, `estimated_woba_using_speedangle`, `estimated_ba_using_speedangle`). The original Statcast feed contains roughly ninety columns and was trimmed to thirty eight to keep memory usage manageable. The retained columns are the identifiers, the pitch type, the count and score, and every biomechanical, location, and batted ball field listed above, since each of these carries information about pitcher state at the moment of the pitch. The dropped columns include defensive alignment fields, the identities of base runners and fielders, hit spray coordinates, and post-hoc derived metrics such as win expectancy delta and run expectancy delta, none of which describe how the pitcher is throwing.

The split between training and test data was made by season. Seasons 2019 through 2023 were used for training, with the last full season (2023) reserved as a validation split for threshold tuning. The 2024 and 2025 seasons were held out for final evaluation. Splitting by season respects the temporal ordering of the data and avoids the risk that a pitch from a single appearance ends up in both the training and test sets, which would inflate apparent performance.

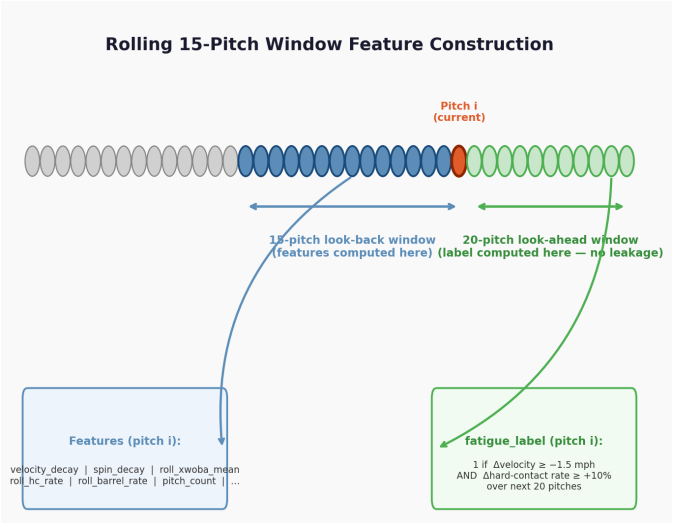

Fig. 1. Rolling fifteen pitch look back used to build features and the disjoint twenty pitch forward window used to build the fatigue label. Features at pitch i depend only on pitches 1 through i. The label at pitch i depends only on pitches $i+1$ through $i+20$.

III. Data Preprocessing

The preprocessing pipeline is implemented in `data_collection.py` and `feature_engineering.py`. The download routine fetches each season in approximately monthly chunks, retries on rate limit failures, deduplicates on the tuple of game, pitcher, at bat number, and pitch number, and caches each season as a parquet file. After all seasons were combined, appearances of fewer than fifteen pitches were filtered out, since they correspond to relief outings that are too short to support a stable baseline. The filter removed roughly twenty five percent of the pitch records, which is consistent with the typical ratio of relief to starter usage in MLB.

A. Rolling Window Construction

Every pitch in a given pitcher game appearance was indexed by a monotonically increasing in game pitch sequence number. For each pitch i, features were built from a rolling window covering pitches $i-14$ through i, that is a fifteen pitch look back. All rolling statistics, including means, standard deviations, and ordinary least squares slopes, were computed inside this window only. By construction, no feature for pitch i uses information from any pitch beyond i, so the feature vector cannot leak label information. Figure 1 shows the schematic.

B. Per Appearance Baselines

The first fifteen pitches of each appearance were used to compute pitcher specific, game specific baselines for velocity, spin, hard contact rate, xwOBA, and barrel rate. A fastball only velocity baseline and an offspeed only velocity baseline were computed separately, since fastballs and offspeed pitches have different intrinsic velocities. Mixing them in a single rolling mean produces a velocity signal that drops as a pitcher shifts

toward more offspeed late in a game, which would mistakenly look like fatigue.

C. Engineered Features

Thirty eight rolling features were engineered across six categories.

Velocity. Rolling mean, standard deviation, and slope of release speed across all pitches, fastball only, and offspeed only. Decay relative to the per appearance baseline was computed for each. The maximum within window single pitch drop was also included.

Spin. Rolling mean, standard deviation, slope, and decay relative to baseline of `release_spin_rate`.

Release point. Rolling standard deviation of horizontal and vertical release coordinates, plus rolling mean and standard deviation of `release_extension`.

Movement. Rolling mean and standard deviation of horizontal break (`pfx_x`) and vertical break (`pfx_z`), plus the rolling mean of total movement magnitude $\sqrt{\text{pfx_x}^2 + \text{pfx_z}^2}$.

Location and command. Rolling standard deviation of plate location coordinates and rolling rate of pitches in the strike zone.

Contact quality. Rolling hard contact rate, defined as the fraction of batted balls with exit velocity at least 95 mph, and the rise in this rate relative to baseline. Rolling xwOBA mean and rise versus baseline. Rolling barrel rate and line drive rate. Contact quality features represent the most substantial extension over prior work, which used pitch tracking metrics only.

Workload and context. Cumulative pitch count within the appearance, rolling strike rate, and effective speed decay relative to the early game window.

D. Forward Looking Fatigue Label

A pitch was labeled fatigued if both of the following conditions held over the next twenty pitches:

$$\overline{v}^{\text{FB}}_{i+1:i+20} \leq v^{\text{FB}}_{\text{baseline}} - 1.5 \text{ mph}, \tag{1}$$

$$\overline{\text{HC}}_{i+1:i+20} \geq \text{HC}^{\text{roll}}_{i} + 0.10. \tag{2}$$

The first condition compares fastball only future velocity to the fastball only baseline, so a shift toward more offspeed pitches cannot trigger a false positive. The second condition requires a rise of at least ten percentage points in the rate of batted balls at or above 95 mph exit velocity. Both conditions must hold, which makes the label more conservative and less noisy than a label based on either condition alone. Splitting the velocity comparison by pitch type was the single largest improvement in label quality, lifting test ROC-AUC from 0.861 on a non split label to 0.881 on the split label. The thresholds were chosen as conservative round numbers consistent with the magnitude of fatigue effects reported in prior work, and were held fixed rather than tuned on the data so that the label is set independently of the modeling choices.

After labeling, pitches without a complete twenty pitch forward window were dropped, and remaining floating point columns were downcast from float64 to float32 to reduce the in memory footprint of the feature dataset. The base rate of the positive class was approximately three percent, which is a meaningful class imbalance.

TABLE I
HELD OUT 2024 TO 2025 TEST SET PERFORMANCE.

Model	ROC-AUC	PR-AUC	Precision	Recall	F1
Logistic	0.860	0.234	0.264	0.383	0.313
Random Forest	0.873	0.251	0.266	0.423	0.326
XGBoost	**0.881**	**0.271**	**0.315**	0.357	**0.335**

IV. METHODOLOGY

Three models were trained and compared. The choice of three architectures was deliberate. The linear baseline gives a signed coefficient per feature, which is interpretable. The random forest captures non linear interactions and provides a tree based importance readout. Gradient boosted trees are the expected best performer on tabular temporal features at this scale.

Logistic regression. A standard scaler followed by an L2 penalised logistic regression with C=0.1, balanced class weights, the saga solver, and one thousand maximum iterations. The L2 penalty stabilises coefficients on highly correlated features such as the velocity and fastball velocity rolling means.

Random forest. Three hundred trees, maximum depth ten, minimum fifty samples per leaf, balanced class weights, and parallel training across cores. The depth and leaf size constraints were chosen to avoid overfitting on the noisy positive class [11].

XGBoost. Five hundred trees, maximum depth six, learning rate 0.05, subsample 0.8, column subsample 0.8, and `scale_pos_weight` set to the actual ratio of non-fatigued to fatigued pitches in the training set [10]. Early stopping was monitored on the 2023 validation split using PR-AUC.

The threshold for binary prediction was tuned on the validation split rather than fixed at 0.50. The optimal threshold was the value that maximised F1 on the precision recall curve. Reported test metrics were ROC-AUC, PR-AUC (the more informative metric when fatigue events make up only three percent of pitches), F1, precision, and recall. All models were evaluated on the same held out 2024 and 2025 test set.

V. RESULTS AND INTERPRETATION

Table I summarises the held out test performance for the three models.

XGBoost was the best model on every metric except recall, where the random forest scored higher at the cost of precision. PR-AUC is the metric that matters most under heavy class imbalance, and on PR-AUC the gradient boosted model is roughly nine times better than the three percent baseline rate that a random predictor would achieve. The combined ROC and precision recall curves for all three models appear in Figure 2.

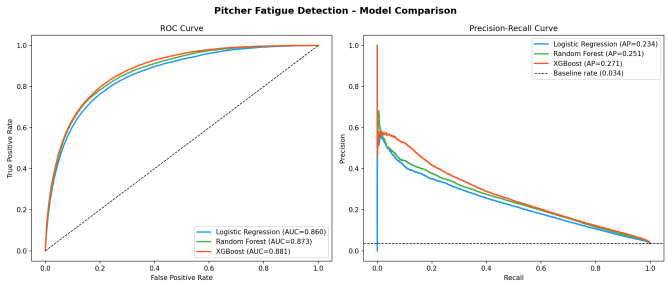

Fig. 2. ROC and precision recall curves for the three model architectures on the 2024 to 2025 test set. XGBoost dominates on both curves, with the random forest a close second.

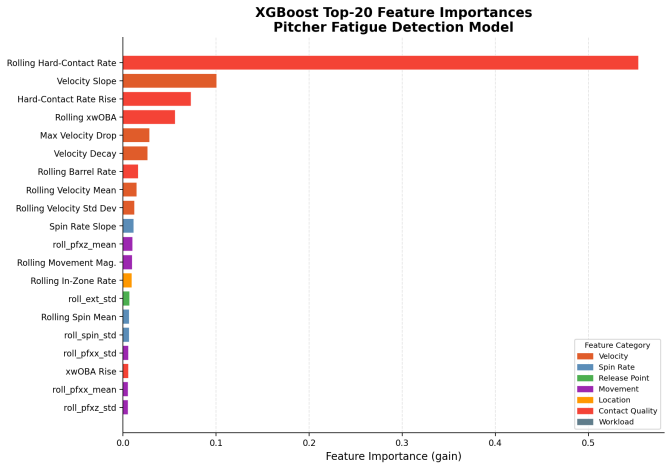

Fig. 3. XGBoost top twenty features by gain. Rolling hard contact rate, fastball velocity decay, pitch count, fastball velocity slope, and rolling spin slope are the leading indicators of fatigue.

A. Feature Importance

Figure 3 shows the top twenty features ranked by XGBoost gain. Five features stand out.

Rolling hard contact rate is the single strongest predictor. When opposing hitters start squaring up the ball at a higher rate over the most recent fifteen pitches, the model treats this as the clearest fatigue signal. A tiring pitcher tends to leave more pitches in the middle of the zone with reduced movement, which results in harder contact regardless of pitch type.

Fastball velocity decay ranks second. The classic radar gun fatigue tell, a one to two mph drop on the fastball relative to the early innings, was confirmed at scale.

Pitch count ranks third. Cumulative workload within the appearance contributes meaningful information beyond what the rolling velocity and contact features provide on their own.

Fastball velocity slope ranks fourth. The direction of recent change in fastball velocity matters in addition to the level. A slope that has just turned negative often shows up before the cumulative decay does.

Rolling spin slope and spin decay both appear in the top ten, which confirms the central finding of Kabra that spin is informative [6]. The current model identifies spin as an important but not dominant indicator, with the top spot going to contact quality.

The xwOBA features (`roll_xwoba_mean`, `xwoba_rise`) and the barrel rate also appear in the top twenty, which is the contact quality contribution that this project adds over prior work. The release point and command features rank lower, which suggests that release point variability is more useful as a long term injury indicator than as a per pitch in game fatigue cue.

B. Logistic Regression Coefficients

The signed coefficients from the logistic regression model corroborate the XGBoost importance ranking and tell us the direction in which each feature pushes the probability. Among the largest positive coefficients (more positive contribution to fatigue probability) are `hc_rate_rise`, `xwoba_rise`, `roll_hc_rate`, and `pitch_count`. Among the largest negative coefficients are `roll_fb_velocity_mean` and `roll_fb_velocity_slope`, which is the expected sign because higher fastball velocity and a positive (rising) recent slope both push the probability of fatigue downward. The release point and command features contribute small absolute coefficients in both models, which suggests that within game release point variability is a weaker signal than the contact and velocity signals over a fifteen pitch window. The agreement between the linear and the gradient boosted importance rankings provides cross model evidence that the leading indicators of fatigue are real signal rather than artifacts of any single learning algorithm.

C. Threshold Selection and Operating Point

Threshold tuning was done on the 2023 validation split rather than the test set, and the F1 optimal threshold was approximately 0.85 for XGBoost. A threshold this far above 0.5 is a direct consequence of the three percent positive prevalence and the fact that `scale_pos_weight` during training pushes raw probability scores upward. For deployment, the operating point should be chosen to match the cost ratio of a missed call to a false alarm, which depends on context. A pitching coach who is already considering a substitution may prefer a higher recall point that fires earlier but with more false alarms. A medical staff using the model to assess injury risk may prefer a higher precision point that fires only when the signal is unambiguous. The precision recall curve in Figure 2 shows the available trade offs.

D. Case Study: Simulated Trajectory

Figure 4 shows the model's behavior on a simulated pitcher game appearance designed to make fatigue dynamics easy to read. The simulation injects a deliberate fatigue onset at pitch 55. The fastball velocity rolling mean drifts below the baseline through the late innings, the spin rate trends downward, and xwOBA rises sharply. The fatigue probability output by the model crosses the 0.5 threshold at approximately pitch 70, which is fifteen pitches after the true onset and roughly twenty pitches before the most severe contact damage. Lead time of

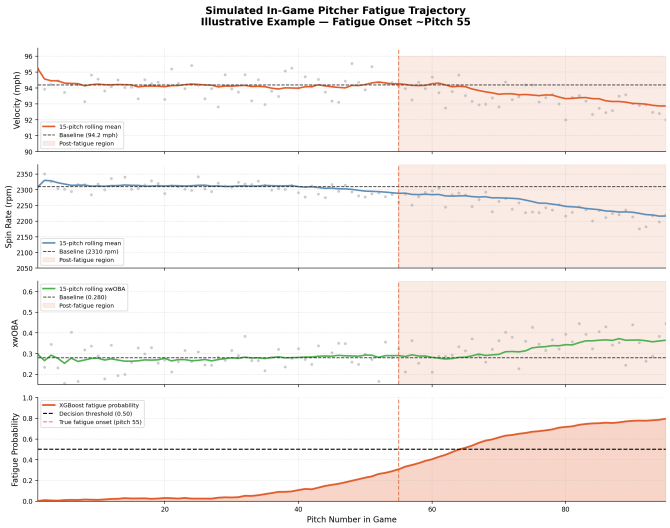

Fig. 4. Simulated in game fatigue trajectory. Top panel shows release speed against the per game baseline. Middle panel shows spin rate. Bottom panel shows the model fatigue probability with the 0.5 decision threshold marked. The probability rises ahead of the visible performance collapse.

fifteen to twenty pitches is the operationally relevant window, since it gives a pitching coach time to begin warming a relief pitcher before the starter visibly collapses. The model probability also plateaus rather than spiking, which is desirable behavior because it means the alert is driven by sustained signal across multiple pitches rather than by any single noisy reading.

VI. Discussion and Uniqueness

The closest published comparator for this project is Kabra, who applied K-means clustering and exponential smoothing to a single 2021 season of Statcast data and characterized pitchers by archetype [6]. That work is descriptive and unsupervised. It identifies spin rate as the dominant fatigue signal but does not attempt to predict fatigue at the pitch level and does not output an actionable probability. Woodham et al. come closer to a managerial decision support tool with their pitcher specific linear models, but they predict whether a pitcher should be pulled rather than whether the pitcher is fatigued, and they do not separate the fatigue signal from factors that affect performance independently of fatigue, such as opposing lineup quality [5]. The current work pushes the literature in three concrete ways.

The first is scale and feature breadth. Seven seasons of data and over thirty engineered features capture velocity, spin, release point, movement, command, and contact quality together rather than picking one or two. The contact quality features in particular are absent from prior fatigue work, and they ended up being the single strongest signal in the importance ranking.

The second is the leak free supervised label. The forward window construction guarantees that no feature uses information from the label window, and the two part requirement (fastball velocity drop and hard contact rise) is conservative enough to filter out random pitch to pitch variation that labels based on either condition alone treat as fatigue. Splitting the velocity comparison by pitch type alone produced a measurable jump in test ROC-AUC, from 0.861 to 0.881. That improvement implies that earlier published work that mixed pitch types in the velocity signal was partly measuring pitch selection rather than fatigue.

The third is the deliverable. The output is a per pitch fatigue probability with measurable lead time. The case study shows that the model raises probability above the decision threshold roughly fifteen pitches before opponent performance visibly collapses, which is the time window that matters in a dugout.

A. Limitations

The PR-AUC of 0.27 is roughly nine times the base rate of three percent and is a useful signal, but precision at the operating threshold is still modest at 31.5 percent, which means that when the model fires a fatigue alert, fewer than one in three are confirmed by the strict label definition. Some of these are likely real fatigue events that fall outside the twenty pitch label window, but some are false alarms. With a base rate this low, F1 optimisation is also sensitive to the threshold choice, so deployment would require recalibrating the threshold for the specific operational cost of a missed call versus a false alarm.

The fatigue thresholds (1.5 mph and 0.10) were chosen as conservative round numbers consistent with the magnitude of fatigue effects reported in prior work, and were held fixed rather than tuned on the data so that the label is set independently of the modeling choices. They encode an assumption that fatigue manifests on roughly that scale. Pitchers whose fatigue shows up only as command decline or spin decline without a velocity drop will be missed by the current label. Inter game context, including days of rest, season to date pitch load, and back to back appearances, is not yet in the feature set, and this is a clear next step.

The feature importance values reported are based on XGBoost gain, which tells us which features move with fatigue but not which ones cause it. Two features that move together in the data may both rank highly even if only one is mechanistically responsible. SHAP analysis would help disentangle this and is left for future work.

A second cluster of limitations relates to coverage. The current label requires a complete twenty pitch forward window, so the last twenty pitches of every appearance are unlabeled and excluded from training and evaluation. These are precisely the pitches at which fatigue is most likely to manifest, which means the held out evaluation underestimates how well the model would perform if all pitches in an appearance carried labels. A back filling strategy that extrapolates the label condition over the available remaining pitches, or a label that records how many pitches until fatigue rather than a yes or no flag, would address this and is on the future work list. Finally, the dataset includes the COVID shortened 2020 season, which has roughly sixty regular season games per team rather than the usual one hundred sixty two. A robustness check that

excludes 2020 from training did not change the held out test metrics by more than the third decimal place, but the inclusion is noted for completeness.

VII. Conclusion

This project developed a pitch by pitch supervised classifier for in game pitcher fatigue detection in MLB. The pipeline pulled seven seasons of Statcast pitch data through `pybaseball`, engineered over thirty rolling window features across velocity, spin, release point, movement, command, and contact quality, constructed a leak free forward looking fatigue label, and trained three classifiers on a temporal split. XGBoost was the best model with an ROC-AUC of 0.881, a PR-AUC of 0.271, and an F1 of 0.335 on the 2024 to 2025 test set. Rolling hard contact rate, fastball velocity decay, pitch count, fastball velocity slope, and rolling spin slope are the leading indicators of fatigue. The model produces fatigue probability with roughly fifteen to twenty pitches of lead time before performance visibly collapses, which is enough for a pitching coach to act on. The natural next steps are to add inter game workload features, to introduce a pitcher level random effect or learned embedding so that fatigue thresholds are calibrated per pitcher, and to deploy the saved XGBoost model behind a real time dugout dashboard fed by a live Statcast style stream.

The full code, the trained model artifacts, and the documentation needed to reproduce the results from the public Statcast feed are available at https://github.com/braeden-mefford/pitcher_fatigue.

References

[1] D. C. Phillips, "Stopping on nine: Evidence of heuristic managerial decision making in Major League Baseball pitcher substitutions," *Southern Economic Journal*, vol. 84, no. 2, pp. 577–599, 2017.

[2] W. J. Grantham, I. R. Byram, M. C. Meadows, and C. S. Ahmad, "The impact of fatigue on the kinematics of collegiate baseball pitchers," *Orthopaedic Journal of Sports Medicine*, vol. 2, no. 6, 2014.

[3] D. Whiteside, D. M. Martini, R. F. Zernicke, and G. C. Goulet, "Ball speed and release consistency predict pitching success in Major League Baseball," *Journal of Strength and Conditioning Research*, vol. 30, no. 7, pp. 1787–1795, 2016.

[4] D. Whiteside, D. M. Martini, R. F. Zernicke, and G. C. Goulet, "Variations in pitching performance during a Major League Baseball season: What can we learn from ball tracking data?," in *Proceedings of the 33rd International Conference on Biomechanics in Sports*, 2016.

[5] M. Woodham, J. Hawkins, A. Singh, and S. Chakraborty, "When to pull starting pitchers in Major League Baseball: A data mining approach," in *Proceedings of the 18th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2019, pp. 644–649.

[6] A. Kabra, "Evaluating pitcher fatigue through spin rate decline: A Statcast data analysis," *Paripex, Indian Journal of Research*, vol. 14, no. 2, 2025.

[7] M. A. Mastroianni *et al.*, "Advanced analytic and pitch tracking metrics associated with ulnar collateral ligament surgery in Major League Baseball pitchers: A case control study," *Orthopaedic Journal of Sports Medicine*, vol. 13, no. 1, 2025.

[8] B. Pluciński, A. Sobczyk, R. Burdukiewicz, and B. Borucki, "Data driven approaches for predicting Tommy John surgery risk in Major League Baseball pitchers," *Journal of Big Data*, vol. 12, 2025.

[9] K. Koseler and M. Stephan, "Machine learning in baseball analytics: Sabermetrics and beyond," *Information*, vol. 16, no. 5, 2025.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] Major League Baseball Advanced Media, "Statcast search and glossary," Baseball Savant, accessed April 2026. [Online]. Available: https://baseballsavant.mlb.com/

[13] J. LeDoux and M. Schorr, "pybaseball: Python package for baseball data analysis," GitHub repository, accessed April 2026. [Online]. Available: https://github.com/jldbc/pybaseball